Sprint 9 - Appointment Booking System Report

Prepared by: Aicha Nait Dayaa

Supervised by: Mr. Abdelmajid Bendrif
Mr. Hamza Bahlaouane

Date: April 3, 2026

Contents

1 Introduction . . . 2
2 Github link . . . 2
3 Documentation . . . 2
3.0.1 Agencies View . . . 2
3.0.2 Appointments Administrative View . . . 2
3.0.3 Advisers . . . 3
3.0.4 User Appointments View . . . 3
3.0.5 Booking Form – step 1 . . . 4
3.0.6 Booking Form – step 2 . . . 5
3.0.7 Booking Form – step 3 . . . 5
3.0.8 Booking Form – step 4 . . . 5
3.0.9 Booking Form – step 5 . . . 6
3.0.10 Booking Form – step 6 . . . 7
3.0.11 Modification Form – step 1 . . . 7
3.0.12 Modification Form – step 2 . . . 8
3.0.13 Modification Form – step 3 . . . 8
3.0.14 Canceled Appointment . . . 9
3.0.15 Confirmation mail . . . 9
3.0.16 Cancelation Confirmation mail . . . 10
4 Module Structure . . . 10
5 Implementation . . . 11
5.0.1 Entity Creation . . . 11
5.0.2 Roles, Vocabulary, and Taxonomy Terms . . . 12
5.0.3 Forms . . . 12
5.0.4 Email Notifications . . . 13
5.0.5 Calendar and Custom Styling . . . 13
5.0.6 Views and Filters . . . 13
5.0.7 Configuration Import with Features . . . 13
5.0.8 Migrations . . . 13
5.0.9 CSV Export with Batch Processing . . . 15
6 Resources . . . 16

1 Introduction

In this sprint, we practiced and applied most of what we learned from the previous sprints. We built a custom module called appointment that allows users to book appointments with advisers at different agencies through a guided multi-step form. The module covers the full lifecycle of an appointment, from booking, to modification and cancellation, while giving administrators a dedicated interface to view, filter, and export appointment data.

2 Github link

Appointment booking system module

3 Documentation

This section provides an overview of the main features of the module through a series of screenshots. It illustrates the system behavior and the available functionalities.

1. Agencies View

The agencies view lists all the agencies available in the system.

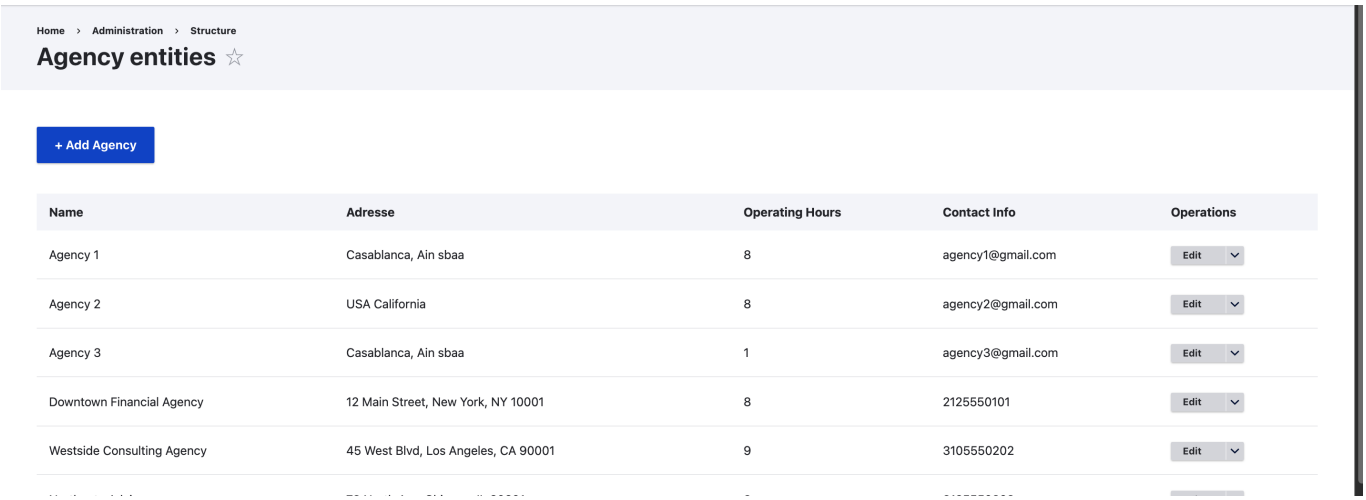

Figure 1: Agecnies view

2. Appointments Administrative View

The admin appointment view gives administrators a full overview of every appointment in the system regardless of who booked it. It includes an **Export as CSV** button that triggers the batch export to export all appointments into a csv file, and an **Add Appointment button** that redirects to the multi-step booking form.

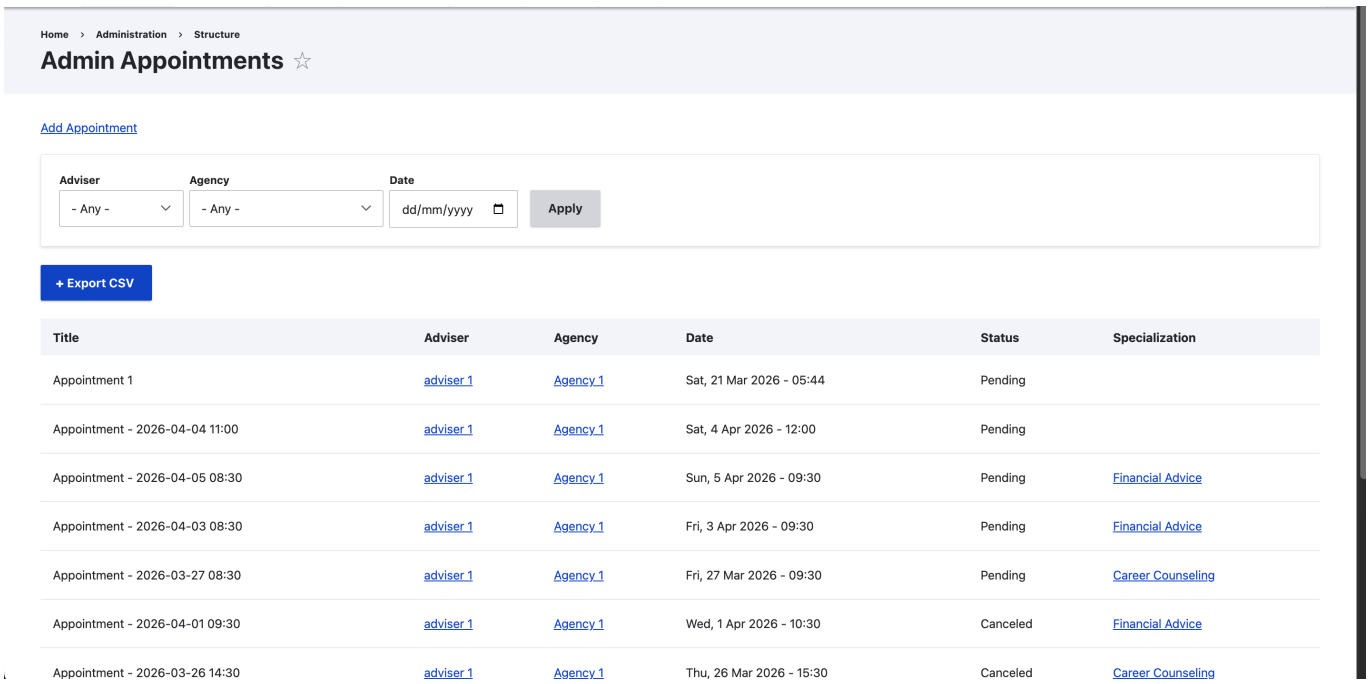

Figure 2: Admin Appointments View

3. Advisers

At **http://appointment-booking-system/admin/people** we can view the adviser users, and can also add adviser users by setting the role to **adviser**.

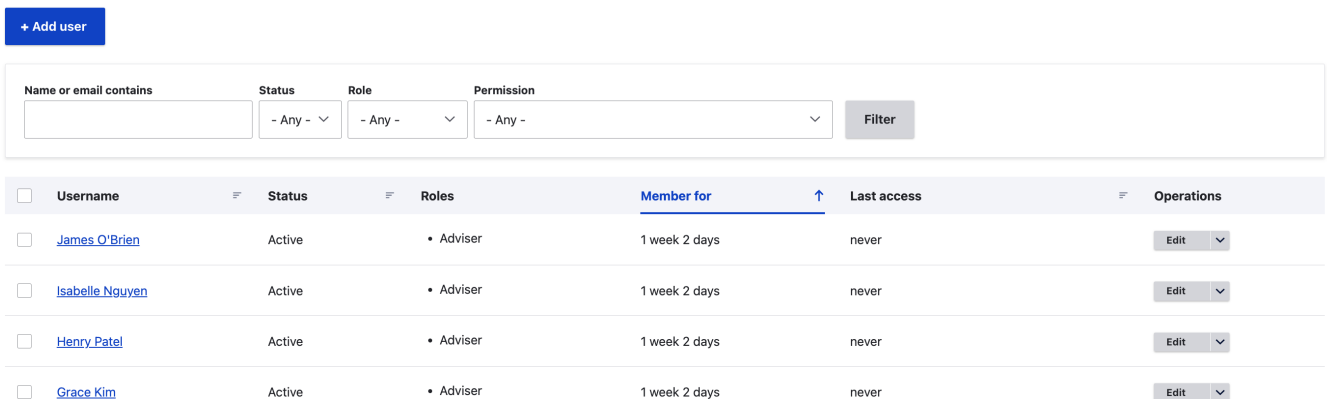

Figure 3: Adviser users

4. User Appointments View

This view lists all appointments of the logged in user, and provides an **Add Appointment** button that redirects to the multi-step booking form.

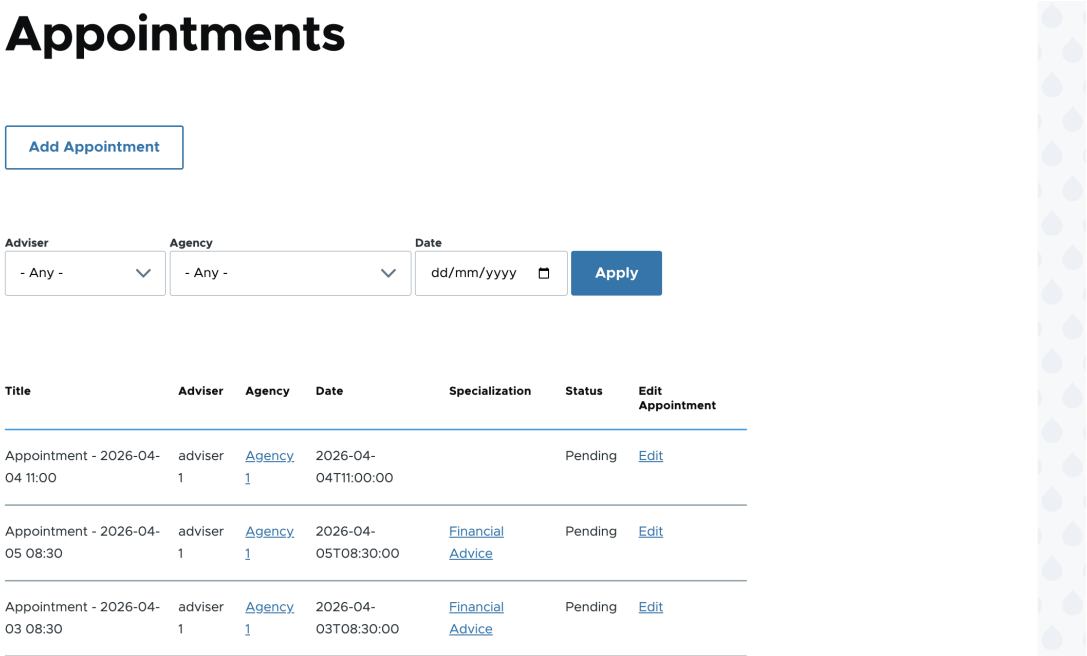

Figure 4: User Appointments View

5. Booking Form – step 1

The first step of the booking form requires selecting an agency from available listed agencies. If an agency is not selected the validation throws an error.

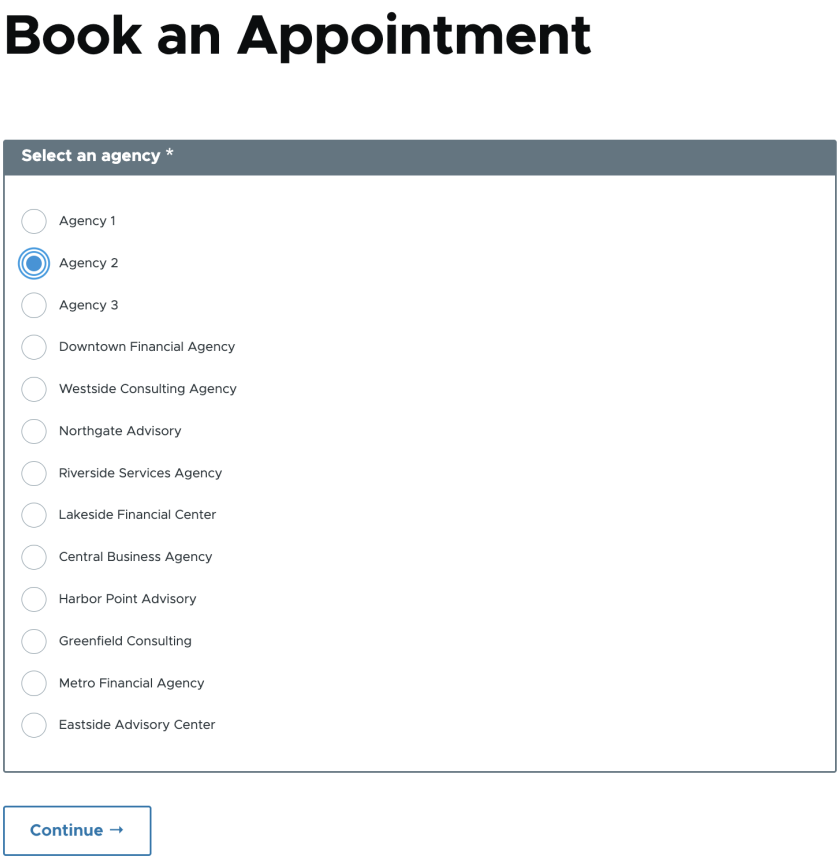

Figure 5: step 1 – selecting an agency

6. Booking Form – step 2

The second step lists all specializations or appointment types that provides the selected agency.

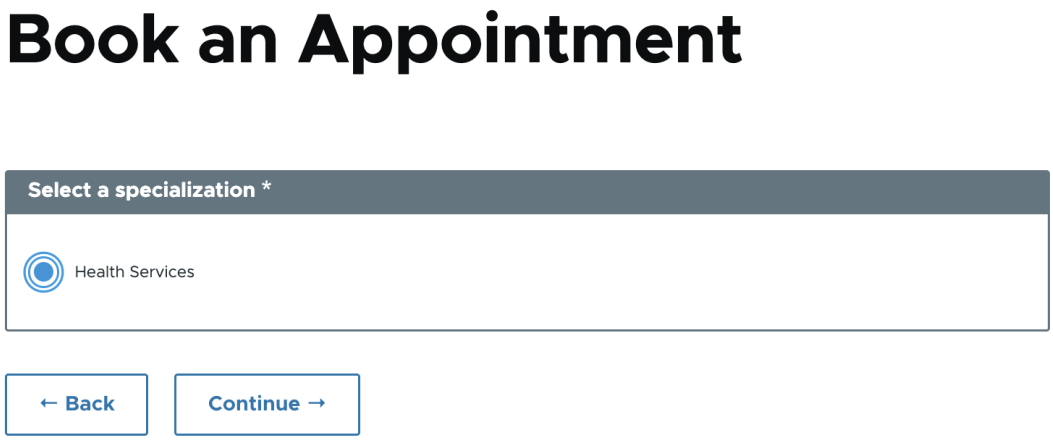

Figure 6: step 2 – selecting an appointment type

7. Booking Form – step 3

The third step of the booking form lists all advisers of the selected agency, specializing in the selected appointment type.

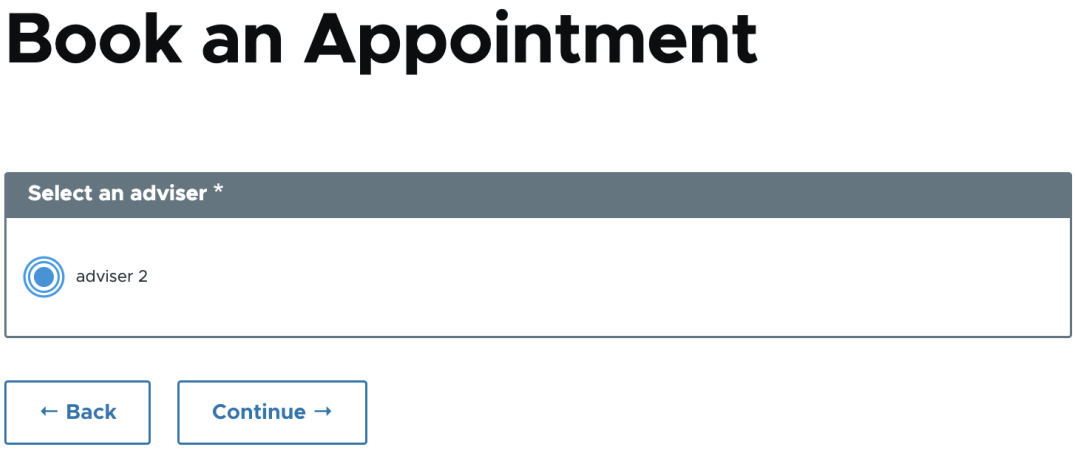

Figure 7: step 3 – selecting an adviser

8. Booking Form – step 4

The fourth step of the form displays the calendar, with pre-checked timeslots for unavailable slots.

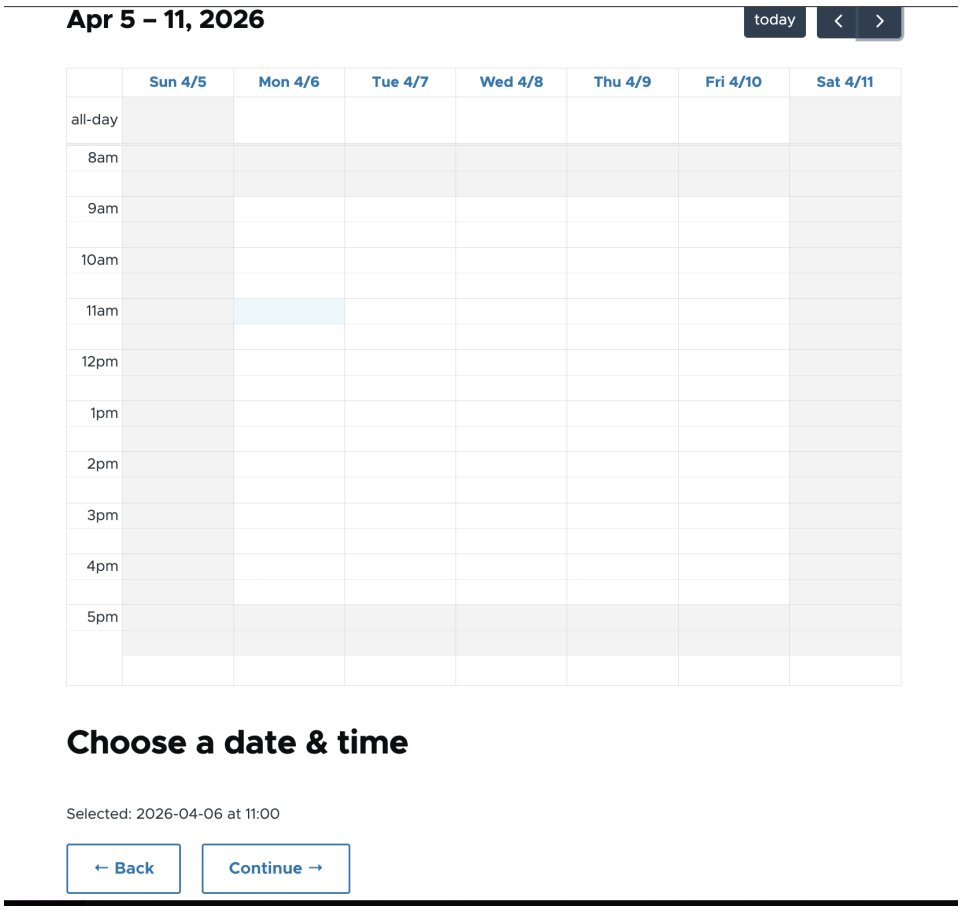

Figure 8: step 4 – selecting a datetime for the appointment

9. Booking Form – step 5

The fifth step of the form asks the user to fill its peronal information.

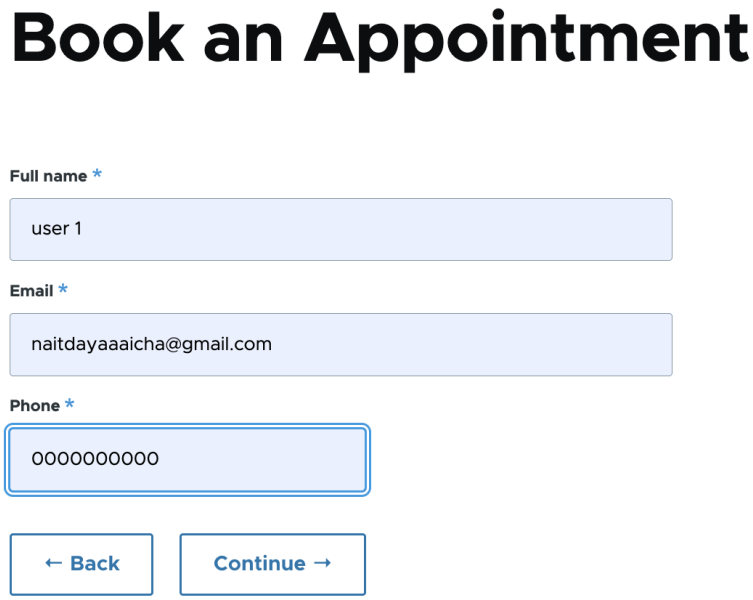

Figure 9: step 5 – Filling user information

10. Booking Form – step 6

The last step of the form, is the confirmation step. It displays the informations entered by the user for last confirmation.

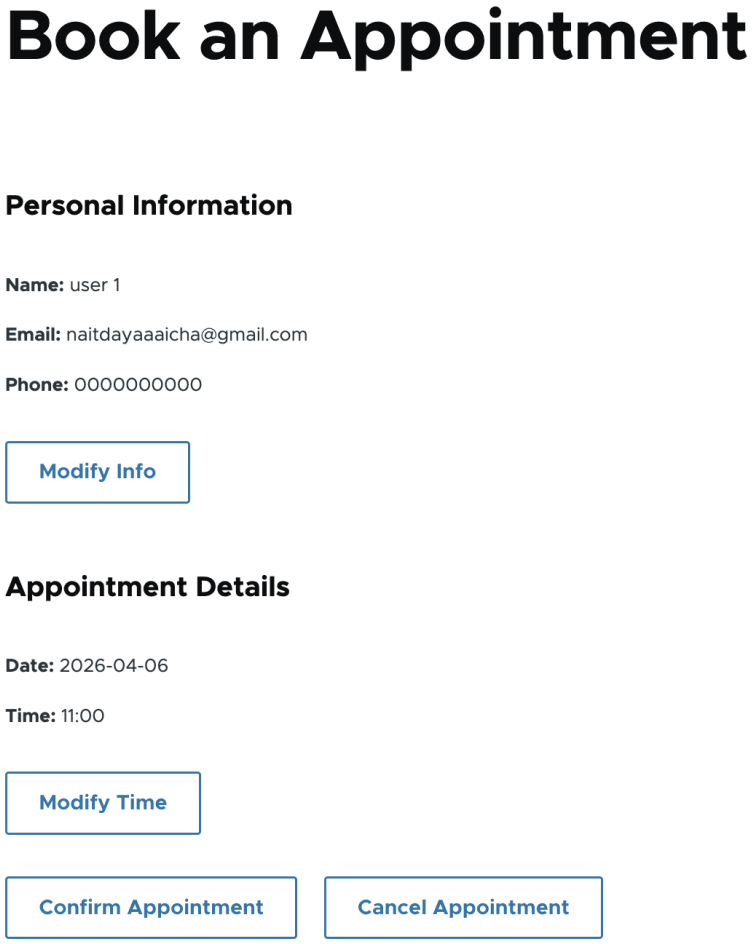

Figure 10: step 6 – Confirmation

11. Modification Form – step 1

Before being able to modify an appointment, an identity verification is required from the user by providing its corresponding phone number

Figure 11: step 1 – verifying phone number

12. Modification Form – step 2

If the phone verification is successful, the user has access to the modification form of the selected appointment.

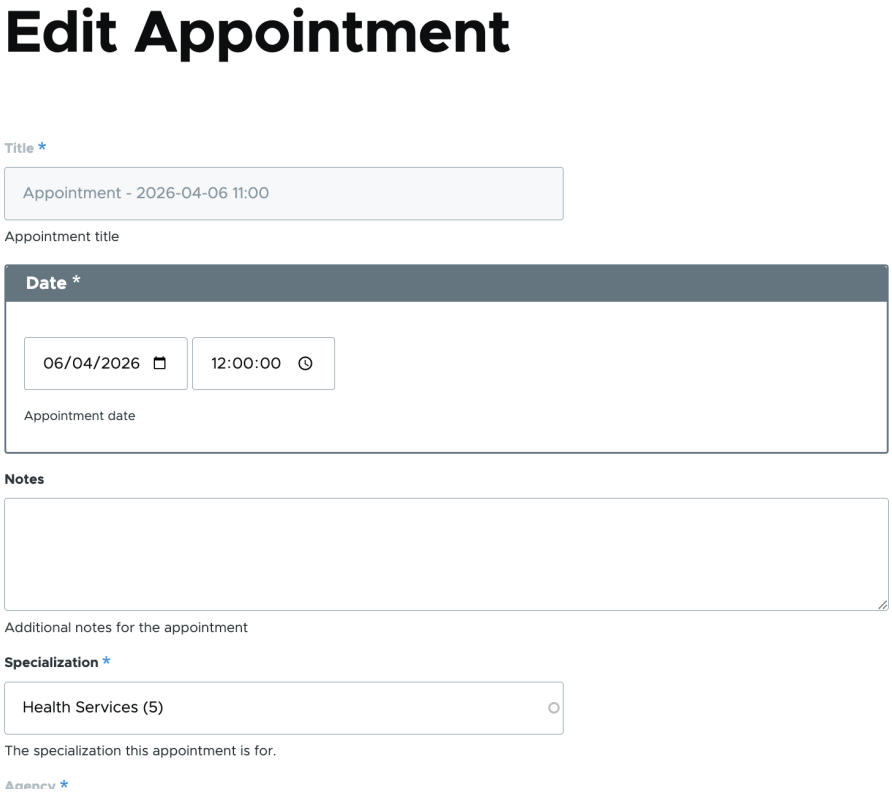

Figure 12: step 2 – Modify Form

13. Modification Form – step 3

The modification form provides two routes, either modifying information, or cancelling the appointment by the two provided buttons at the bottom of the form.

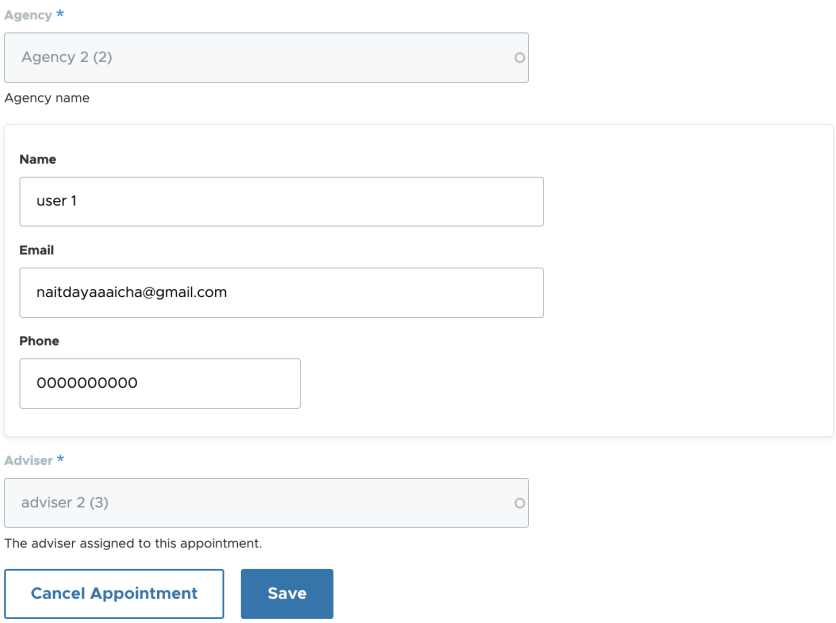

Figure 13: step 2 – Modify Form

14. Canceled Appointment

Once the appointment is canceled, the user can't modify it, and it is only soft deleted.

Appointment - 2026-04-06 11:00	adviser 2	Agency 2	2026-04-06T11:00:00	Health Services	Canceled

Figure 14: Cancelled Appointment

15. Confirmation mail

Once the appointment is booked, the user receives a confirmation email.

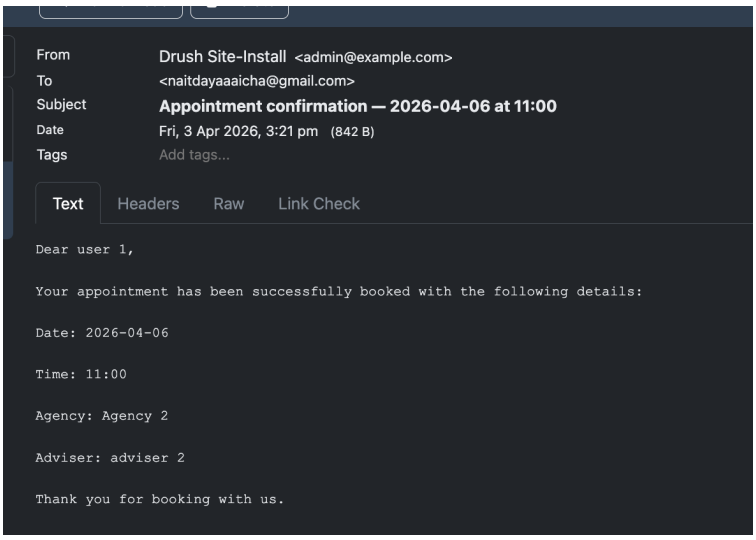

Figure 15: Confirmation mail

16. Cancelation Confirmation mail

Once the appointment is canceled, the user receives a cancelation confirmation email.

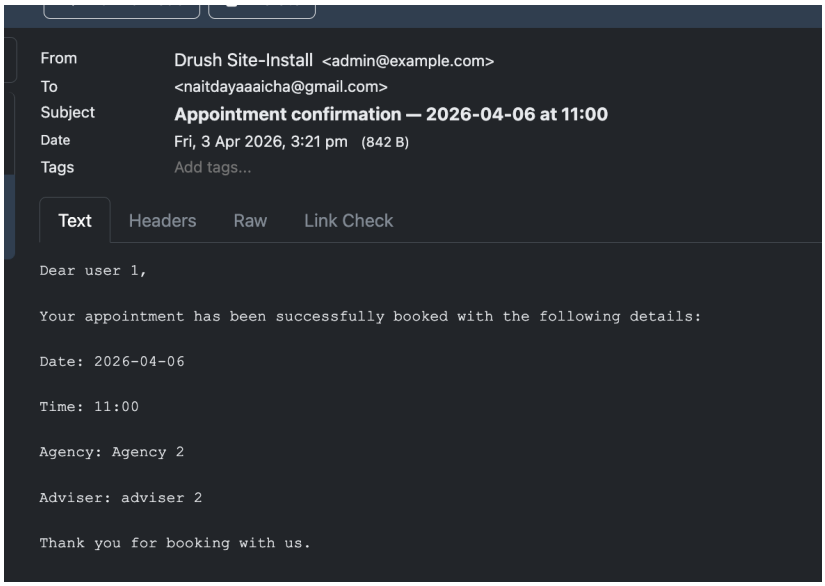

Figure 16: Cancel Confirmation mail

4 Module Structure

The image below illustrates the module structure

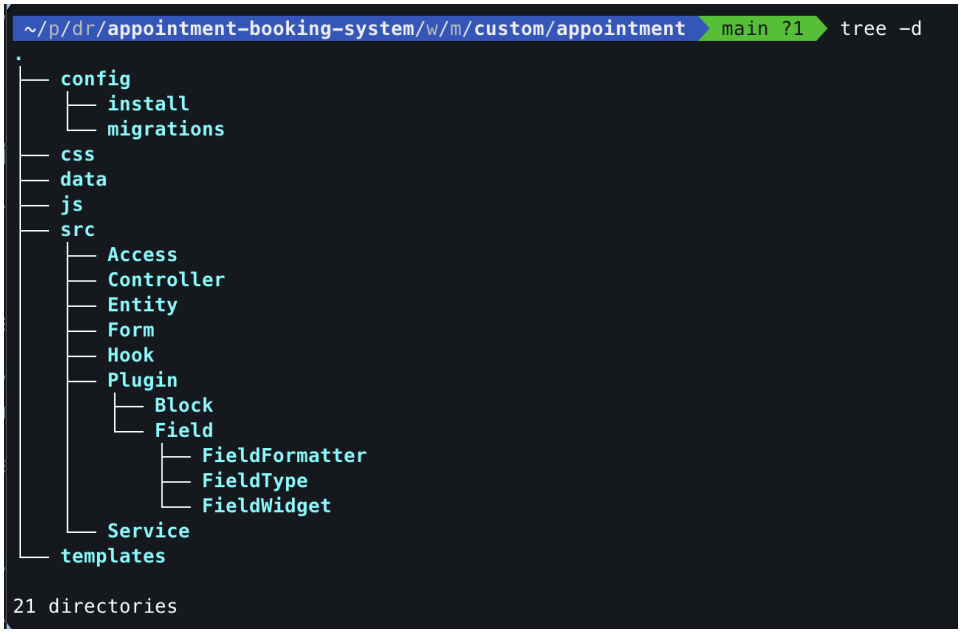

Figure 17: Project Structure

5 Implementation

In this section, we go through the main steps taken to build the module.

1. Entity Creation

We created two custom content entities – `appointment` and `agency` – each with their own access control handler to manage permissions for viewing, creating, editing, and deleting. The adviser is not a separate entity but an extension of the Drupal user entity, with additional fields added through the hook **hook_entity_base_field_info()**.

```
#[ContentEntityType(
  id: 'appointment_entity',
  label: new TranslatableMarkup('Appointment entity'),
  label_collection: new TranslatableMarkup('Appointment entities'),
  entity_keys: [
    'id' => 'id',
    'label' => 'title',
    'uuid' => 'uuid',
    'owner' => 'uid',
    'langcode' => 'langcode',
  ],
  handlers: [
    'access' => AppointmentAccessControlHandler::class,
    'list_builder' => AppointmentListBuilder::class,
    'view_builder' => EntityViewBuilder::class,
    'views_data' => EntityViewsData::class,
    'translation' => ContentTranslationHandler::class,
    'route_provider' => [
      'html' => 'Drupal\Core\Entity\Routing\AdminHtmlRouteProvider',
    ],
    'form' => [
      'default' => ContentEntityForm::class,
      'edit_booking' => AppointmentEditBookingForm::class,
      'delete'  => ContentEntityDeleteForm::class,
    ],
  ],
  links: [
    'canonical' => "/appointment/{appointment_entity}",
    'add-form' => "/appointment/add",
    'edit-form' => "/appointment/{appointment_entity}/edit",
    'delete-form' => "/appointment/{appointment_entity}/delete",
    'collection' => "/admin/structure/appointments",
  ],
```

```
  admin_permission: 'administer appointment entities',
  base_table: 'appointment',
  data_table: 'appointment_entity_data',
  translatable: true,
  field_ui_base_route: 'appointment_entity.settings',
)]
```

2. Roles, Vocabulary, and Taxonomy Terms

The adviser role, the **appointment_type** vocabulary, and its default taxonomy terms were all created programmatically in the **.install** file using the hook **hook_install()**, so they are set up automatically when the module is installed.

github link

3. Forms

For the administration side, the add and edit forms for both entities use Drupal's built-in **ContentEntityForm**. The entity fields were configured using Field UI, which makes the entities fieldable and allows administrators to manage fields from the UI.

For the user-facing booking form, we extended `FormBase` to build a custom multi-step form. Since the form does not refresh between steps, we used `$form_state` to store data across steps. As a future improvement, we would replace this with `TempStore` to prevent data loss if the page is accidentally refreshed before submission.

```
public function buildForm(array $form, FormStateInterface $form_state) {
  $step = $form_state->get('step') ?? 1;

  switch ($step) {
    case 1:
      // step 1 implmentation
    case 2:
      // step 2 implmentation
    case 3:
      // step 3 implmentation
    case 4:
      // step 4 implmentation
    case 5:
      // step 5 implmentation
    case 6:
      // step 6 implmentation
  }
}
```

4. Email Notifications

Confirmation emails are sent automatically after a booking is successfully submitted. Additional emails are triggered when an appointment is modified or cancelled, keeping the user informed at every stage of the process.

5. Calendar and Custom Styling

The date and time selection in Step 4 uses FullCalendar, integrated through Drupal Behaviors in the module's JavaScript file. We also added a custom field with its own CSS to display appointment information in a styled format on the confirmation step.

github link

6. Views and Filters

The admin view and the user view were both created using the Views module. The custom filters were added through a **hook_form_FORM_ID_alter** and **hook_views_query_alter** implementations in the **.module** file.

github link

7. Configuration Import with Features

All the configuration built through the UI — including views, field settings, and display modes — was exported and packaged using the Features module. This allows the module to be installed on a new Drupal instance with all configuration applied automatically without any manual setup.

8. Migrations

We created and tested migrations for the three main data sets: agencies, advisers, and appointments.

```yaml
id: appointments_import
label: Appointments
migration_group: appointment
migration_dependencies:
  required:
      - agencies_import
      - advisers_import

source:
  plugin: csv
  path: 'modules/custom/appointment/data/appointments.csv'
  header_row_count: 1
  ids:
    - ID
```

```yaml
  fields:
    - name: ID
      label: 'ID'
    - name: title
      label: 'Title'
    - name: date
      label: 'Date'
    - name: agency_id
      label: 'Agency ID'
    - name: adviser_id
      label: 'Adviser ID'
    - name: specialization
      label: 'Specialization'
    - name: customer_name
      label: 'Customer Name'
    - name: customer_email
      label: 'Customer Email'
    - name: customer_phone
      label: 'Customer Phone'
    - name: status
      label: 'Status'

destination:
  plugin: entity:appointment_entity

process:
  title: title
  date: date
  status: status

  agency:
    plugin: migration_lookup
    migration: agencies_import
    source: agency_id

  adviser:
    plugin: migration_lookup
    migration: advisers_import
    source: adviser_id

  specialization:
    plugin: entity_lookup
    source: specialization
    entity_type: taxonomy_term
    value_key: name
```

```
    bundle_key: vid
    bundle: appointment_type
    ignore_case: true
    destination_field: tid

  customer_info/phone: customer_phone
  customer_info/email: customer_email
  customer_info/name: customer_name
```

Migration	Machine Name	Status	Total	Imported	Unprocessed	Messages	Last Imported	Operations
Advisers	advisers_import	Idle	10	10	0	0	2026-03-25 11:14:05	Execute
Agencies	agencies_import	Idle	10	10	0	0	2026-03-25 11:13:54	Execute
Appointments	appointments_import	Idle	10	10	0	0	2026-03-25 11:14:19	Execute

9. CSV Export with Batch Processing

The CSV export is triggered from the admin appointment view and uses Drupal's Batch API to process appointments in chunks. This ensures the export works reliably even with a large number of records without running into memory limits.

github link

6 Resources

- Batch Processing
- FullCalendar Documentation
- Loom video link (video 1)
- Loom video link (video 2)